

НИУ ИТМО

Лабораторная работа №6

Дисциплина: Методы оптимизации

Тема: «Методы поиска глобального экстремума»

Выполнили: студенты группы Р3209

Ведерникова А.В.

Исаева А.А.

Проверила: Селина Е.Г.

2026 г.

1 Решение задачи коммивояжера с помощью генетического алгоритма

Цель задания: Найти кратчайший замкнутый маршрут, проходящий через все города, используя генетический алгоритм.

1.1 Пример промежуточных расчетов

Размер популяции выбран равным 4. Оператор мутации представляет собой случайную перестановку двух чисел в хромосоме, выбранных случайно по равномерному закону. Вероятность мутации составляет 0.01.

Ниже представлены таблицы с пошаговым решением этапа инициализации, скрещивания и отбора.

Таблица 1: Исходная популяция

№ строки	Код хромосомы	Значение целевой функции
1	12345	29
2	21435	29
3	54312	32
4	43125	32

Для скрещивания были выбраны следующие пары родителей: (1, 3) и (2, 4).

Таблица 2: Результаты скрещивания и полученные потомки

№ строки	Родители	Потомки	Значение ЦФ для потомков
1	1/23/45	543 12	32
3	5/43/12	1/23/54 (после мутации: 13254)	28
2	21435	43125	32
4	43125	21435	29

Для потомка (12354) сработал оператор мутации, в результате чего обменялись местами числа 2 и 3. Хромосома приняла вид (13254). Популяция первого поколения после отсечения худших особей представлена в таблице 3.

Таблица 3: Популяция первого поколения после отбора

№ строки	Код	Значение ЦФ	Вероятность участия в размножении
1 (1)	12345	29	28/111
2 (2)	21435	29	28/111
3 (H)	13254	28	29/111
4 ()	21435	29	28/111

2 Поиск кратчайшего пути в графе с помощью алгоритма муравьиной колонии

```
1 import random
2 import math
3
4 dist = [
5     [0, 10, 8, 1, 9],
6     [10, 0, 5, 8, 5],
7     [8, 5, 0, 10, 3],
8     [1, 8, 10, 0, 1],
9     [9, 5, 3, 1, 0]
10 ]
11
12 POP_SIZE = 4
13 MUTATION_RATE = 0.01
14 GENERATIONS = 100
15 ELITE_SIZE = 1
16
17 def route_length(route):
18     total = 0
19     for i in range(len(route) - 1):
20         total += dist[route[i]][route[i + 1]]
21     total += dist[route[-1]][route[0]]
22     return total
23
24 def fitness(route):
25     return 1 / route_length(route)
26
27 def create_route():
28     route = list(range(len(dist)))
29     random.shuffle(route)
30     return route
31
32 def selection(population):
33     total_fit = sum(fitness(r) for r in population)
34     pick = random.uniform(0, total_fit)
35     current = 0
36
37     for route in population:
38         current += fitness(route)
39         if current >= pick:
40             return route
41
42 def fill_child(child, parent, start, end):
43     n = len(parent)
44     parent_index = (start + 1) % n
45     values = []
46
47     for _ in range(n):
48         city = parent[parent_index]
49
50         if city not in child:
51             values.append(city)
52
53         parent_index = (parent_index + 1) % n
54
55     value_index = 0
56
```

```

57     for i in range(n):
58         if child[i] is None:
59             child[i] = values[value_index]
60             value_index += 1
61
62 def crossover(parent1, parent2):
63     n = len(parent1)
64
65     while True:
66         start, end = sorted(random.sample(range(1, n), 2))
67         if end - start >= 2:
68             break
69
70     child1 = [None] * n
71     child2 = [None] * n
72
73     child1[start:end] = parent2[start:end]
74     child2[start:end] = parent1[start:end]
75
76     fill_child(child1, parent1, start, end)
77     fill_child(child2, parent2, start, end)
78
79     return child1, child2
80
81 def mutate(route):
82     if random.random() < MUTATION_RATE:
83         i, j = random.sample(range(len(route)), 2)
84         route[i], route[j] = route[j], route[i]
85     return route
86
87 def genetic_algorithm():
88     population = [create_route() for _ in range(POP_SIZE)]
89
90     for generation in range(GENERATIONS):
91         population = sorted(population, key=route_length)
92
93         new_population = population[:ELITE_SIZE]
94
95         while len(new_population) < POP_SIZE:
96             p1 = selection(population)
97             p2 = selection(population)
98
99             child1, child2 = crossover(p1, p2)
100
101             child1 = mutate(child1)
102             child2 = mutate(child2)
103
104             new_population.append(child1)
105
106             if len(new_population) < POP_SIZE:
107                 new_population.append(child2)
108
109             population = new_population
110
111     best = min(population, key=route_length)
112     return best, route_length(best)
113
114 best_route, best_distance = genetic_algorithm()
115
116 print("Генетический алгоритм")

```

```

117 print("Лучший маршрут:", " -> ".join(f"Город {i + 1}" for i in best_route)
    , "->", f"Город {best_route[0] + 1}")
118 print("Длина маршрута:", best_distance)
119
120
121 graph = {
122     "A": {"B": 13, "C": 7, "F": 7},
123     "B": {"A": 13},
124     "C": {"A": 7, "D": 8, "E": 8},
125     "D": {"C": 8, "E": 11, "G": 11},
126     "E": {"C": 8, "D": 11, "G": 19},
127     "F": {"A": 7, "G": 38},
128     "G": {"D": 11, "E": 19, "F": 38}
129 }
130
131 ANTS = 20
132 ITERATIONS = 100
133 ALPHA = 1
134 BETA = 2
135 EVAPORATION = 0.5
136 Q = 100
137 START = "A"
138 END = "G"
139
140 pheromone = {}
141 for u in graph:
142     for v in graph[u]:
143         pheromone[(u, v)] = 1.0
144
145 def choose_next(current, visited):
146     variants = []
147
148     for neighbor, distance in graph[current].items():
149         if neighbor not in visited:
150             tau = pheromone[(current, neighbor)] ** ALPHA
151             eta = (1 / distance) ** BETA
152             variants.append((neighbor, tau * eta))
153
154     if not variants:
155         return None
156
157     total = sum(value for _, value in variants)
158     r = random.uniform(0, total)
159     current_sum = 0
160
161     for neighbor, value in variants:
162         current_sum += value
163         if current_sum >= r:
164             return neighbor
165
166 def path_length(path):
167     return sum(graph[path[i]][path[i + 1]] for i in range(len(path) - 1))
168
169 def ant_colony():
170     best_path = None
171     best_len = math.inf
172
173     global pheromone
174
175     for iteration in range(ITERATIONS):

```

```

176     all_paths = []
177
178     for ant in range(ANTS):
179         current = START
180         visited = {START}
181         path = [START]
182
183         while current != END:
184             next_node = choose_next(current, visited)
185
186             if next_node is None:
187                 break
188
189             path.append(next_node)
190             visited.add(next_node)
191             current = next_node
192
193             if path[-1] == END:
194                 length = path_length(path)
195                 all_paths.append((path, length))
196
197                 if length < best_len:
198                     best_len = length
199                     best_path = path
200
201         for edge in pheromone:
202             pheromone[edge] *= (1 - EVAPORATION)
203
204         for path, length in all_paths:
205             deposit = Q / length
206             for i in range(len(path) - 1):
207                 u = path[i]
208                 v = path[i + 1]
209                 pheromone[(u, v)] += deposit
210                 pheromone[(v, u)] += deposit
211
212     return best_path, best_len
213
214 aco_path, aco_len = ant_colony()
215
216 print("\nМуравьиная колония")
217 print("Лучший путь:", " -> ".join(aco_path))
218 print("Длина пути:", aco_len)

```

Листинг 1: Реализация алгоритмов оптимизации

3 Заключение

В ходе выполнения лабораторной работы были изучены и практически применены методы поиска глобального экстремума на примере двух классических задач дискретной оптимизации.

Таким образом, оба подхода доказали свою применимость и высокую эффективность при решении сложных комбинаторных задач, для которых точные методы полного перебора являются вычислительно неэффективными или неприменимыми на больших объемах данных.